

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	20% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	S.Krishna Chaithanya	Reg. No.:	192424287
Section / Batch:	B.Tech-AIDS/2024	Date:	

Code of Conduct

I S.Krishna Chaithanya(192424287) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S.Krishna Chaithanya

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.
2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing. Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.
3. An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques. Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.
4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames. Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.
5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing. Analyze the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

1. CFG Trees vs Dependency Parsing

Aspect	CFG Trees	Dependency Parsing
Basic representation	Hierarchical phrase structure	Word-to-word relationships
Main units	Noun Phrase (NP), Verb Phrase (VP), etc.	Head and dependent words
Focus	Sentence constituents	Syntactic relationships
Example	$S \rightarrow NP VP$	saw $\rightarrow$ She, saw $\rightarrow$ man
Long-distance relationships	Less direct	Captured directly
Suitability	Grammar and phrase analysis	Relationship and syntactic analysis

Analysis:

CFG represents a sentence as nested constituents. For example, "*The student solved the problem*" can be represented as $S \rightarrow NP VP$, where NP is "*The student*" and VP is "*solved the problem*." This is useful for grammar checking and identifying phrases.

Dependency parsing directly connects words according to their grammatical relationships. For example, "*solved*" can be the head of "*student*" and "*problem*." This makes relationships between individual words easier to analyze.

Justification: Dependency parsing is more effective for capturing relationships between words, especially in applications such as information extraction, question answering, and semantic analysis.

CFG is preferable when detailed phrase structure is required.

Conclusion:

Dependency parsing $\rightarrow$ better for word-level relationships.

CFG $\rightarrow$ better for constituent/phrase-level structure.

2. Top-Down Parsing vs Earley Parsing

Aspect	Top-Down Parsing	Earley Parsing
Parsing direction	Starts from start symbol	Processes input left-to-right
Strategy	Predicts grammar rules	Uses prediction, scanning and completion
Ambiguity	Can be inefficient	Handles ambiguity naturally
Incomplete input	Poorer support	Better support
Left recursion	Can cause problems	Handles left recursion
Complexity	Depends heavily on grammar	General CFG: $O(n^3)$ worst case
Best use	Simple, predictable grammars	General and dynamic input

Analysis:

Top-down parsing begins with the start symbol and tries to generate the input. It can be efficient for simple grammars, but it may perform unnecessary predictions and can have problems with left-recursive grammars.

Earley parsing maintains a set of states while reading the input. It uses three major operations:

1. Prediction – predicts possible grammar rules.
2. Scanning – matches the next input token.
3. Completion – completes partially recognized rules.

This allows Earley parsing to handle ambiguous, incomplete, and left-recursive grammars more effectively.

Justification: For a real-time application receiving dynamic user input, Earley parsing is more suitable because it can maintain partial parses and handle ambiguity without requiring major grammar modifications.

Conclusion:

Earley parsing is more suitable for dynamic and incomplete input, although optimized top-down parsers can be faster for restricted, deterministic grammars.

3. CFG vs PCFG vs Neural Parsing for Ambiguity

Consider:

“She saw the man with a telescope.”

This sentence has at least two interpretations:

- She used a telescope to see the man.
- She saw a man who had a telescope.

Approach	Ambiguity Handling	Strength	Limitation
CFG	Generates multiple valid parse trees	Simple and interpretable	Cannot naturally rank interpretations
PCFG	Assigns probabilities to parse trees	Can select the most probable parse	Depends on grammar and training probabilities
Neural Parser	Learns contextual patterns from data	Strong contextual understanding	Requires large training data and computational resources

CFG

A CFG may generate both interpretations because both structures satisfy the grammar. However, standard CFG does not provide enough information to determine which interpretation is more likely.

PCFG

A Probabilistic CFG assigns probabilities to grammar rules. Therefore, it can rank competing parse trees and choose the one with the highest probability.

Neural Parsing

Neural parsers use contextual representations learned from large datasets. They can consider surrounding words and linguistic patterns when deciding the intended structure.

Justification: Neural parsing is generally the most effective for real-world NLP applications, because ambiguity depends heavily on context and usage patterns. PCFG is useful when interpretability and relatively simple probabilistic modeling are important.

Conclusion:

Neural parsing → best overall for modern real-world applications.

PCFG → good balance between probabilistic reasoning and interpretability.

CFG → useful for basic grammatical structure but weak at ambiguity resolution.

4. Feature Structures vs Subcategorization Frames

Aspect	Feature Structures	Subcategorization Frames
Purpose	Represents grammatical features	Represents arguments required by verbs
Handles	Number, gender, tense, person, case	Verb complements and argument structure
Agreement	Excellent	Limited
Verb argument requirements	Can represent them, but indirectly	Excellent
Example	Subject.number = plural	give → NP + NP + NP/PP
Main use	Grammatical constraints	Verb valency

Feature Structures

Feature structures represent grammatical properties as attributes and values.

For example:

Subject:

person = 3
number = singular

Verb:

person = 3
number = singular

Thus:

She walks. ✓

She walk. ✗

The system can enforce subject–verb agreement using matching features.

Subcategorization Frames

Subcategorization describes which arguments a verb requires.

For example:

eat → NP

give → NP NP

depend → PP

For:

John gave Mary a book.

The verb *gave* requires a structure involving a subject and appropriate complements.

Justification: Feature structures provide better support for enforcing grammatical correctness because they can explicitly encode agreement features such as person, number, tense, and case. Subcategorization frames are particularly valuable for ensuring that verbs receive the appropriate arguments.

Conclusion:

For subject–verb agreement → Feature Structures are superior.

For verb argument structure → Subcategorization Frames are superior.

For a complete grammar system, using both together is the strongest solution

5. Transition-Based vs Graph-Based Dependency Parsing

Aspect	Transition-Based Parsing	Graph-Based Parsing
Decision method	Makes local sequential decisions	Searches for best global dependency tree
Processing	Step-by-step transitions	Scores possible dependency structures
Speed	Usually very fast	Usually slower
Global information	Limited	Strong
Error propagation	Higher	Lower
Large datasets	Excellent	More computationally expensive
Accuracy	High with modern models	Often strong due to global optimization

Transition-Based Parsing

Transition-based parsing builds a dependency tree through a sequence of actions such as:

SHIFT

REDUCE

LEFT-ARC

RIGHT-ARC

Each decision is made based on the current parser state.

Advantages:

- Very fast.
- Low computational cost.
- Suitable for real-time applications.
- Scales well to large datasets.

Disadvantage: An early incorrect decision can affect later decisions, causing error propagation.

Graph-Based Parsing

Graph-based parsing considers possible dependency arcs and attempts to find the highest-scoring complete dependency tree.

Instead of making only local decisions, it considers global tree structure.

Advantages:

- Better global consistency.
- Less dependent on a sequence of local decisions.
- Can produce highly accurate parses.

Disadvantages:

- More computationally expensive.
- Generally slower for very large datasets.

Justification

For a large-scale NLP system requiring fast dependency parsing, transition-based parsing is generally more suitable because of its speed and relatively low computational requirements.

For applications where maximum parsing accuracy and global structural consistency are more important than speed, graph-based parsing can be preferable.

Conclusion:
Large-scale + real-time → Transition-based parsing.
Accuracy + global optimization → Graph-based parsing.

Overall Comparison				
Question		Recommended Approach		Main Reason
CFG vs Dependency		Dependency Parsing		Better word-to-word relationships
Top-Down vs Earley		Earley Parsing		Handles incomplete and ambiguous input
CFG vs PCFG vs Neural		Neural Parsing		Strong contextual ambiguity resolution
Feature Structures vs Subcategorization		Feature Structures + Subcategorization		Agreement + argument constraints
Transition vs Graph		Transition-Based		Faster and scalable
Key Metrics / Performance View				
Technique	Speed	Ambiguity Handling	Global Context	Scalability
CFG	Medium	Low	Low	Medium
PCFG	Medium	Medium–High	Medium	Medium
Neural Parsing	Medium–High*	High	High	High*
Earley	Medium	High	Medium	Medium
Transition-Based	Very High	Medium	Low–Medium	Very High
Graph-Based	Low–Medium	High	High	Medium

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____